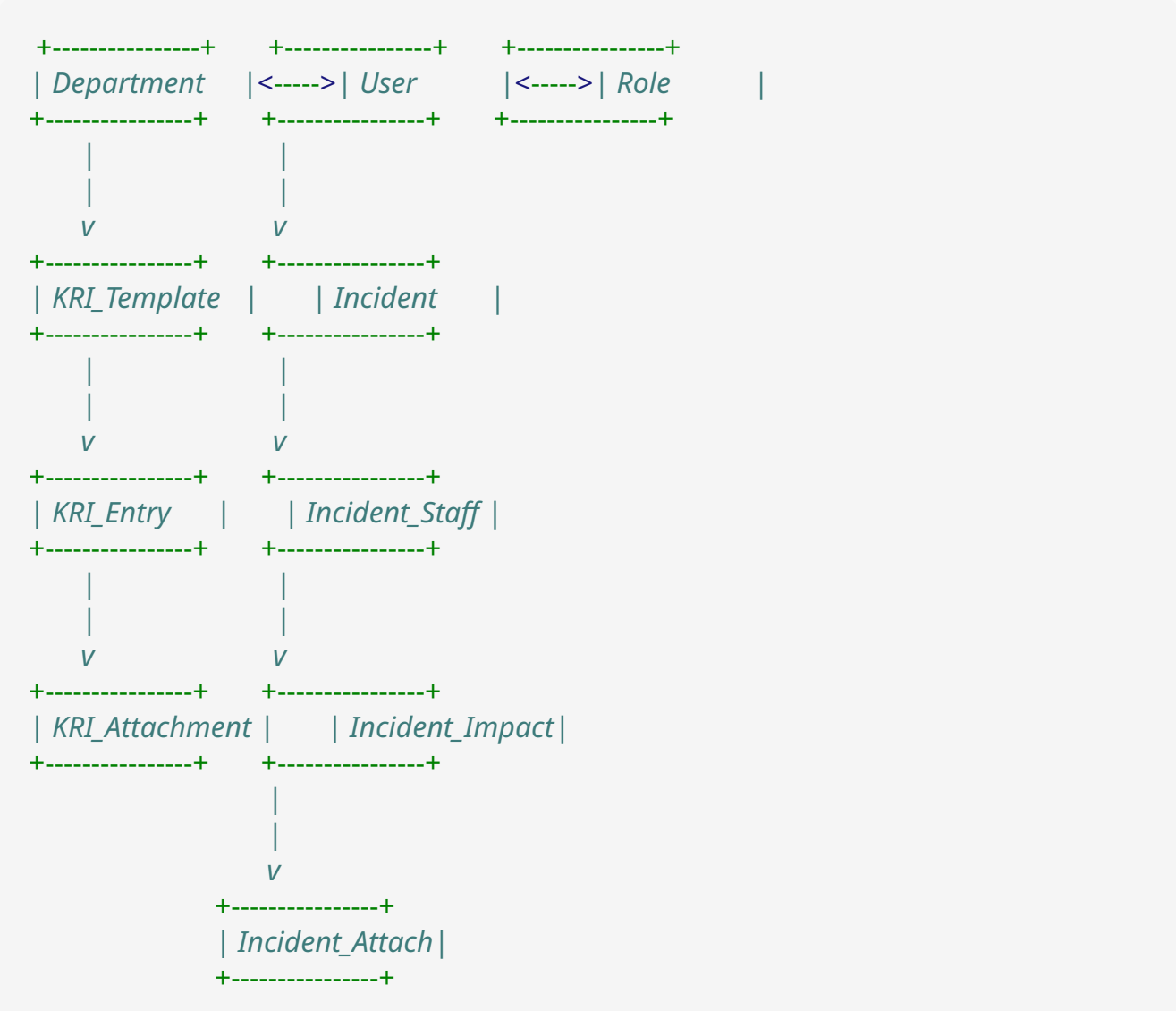
Database Schema Design for Operational Risk Management Application

Overview

This database schema is designed to support the Operational Risk Management application with two primary modules: KRI Reporting and Incident Reporting. The schema follows relational database principles and includes necessary tables, relationships, and constraints to meet the requirements specified in the BRD.

Entity Relationship Diagram (Conceptual)



Detailed Schema

1. User Management Tables

Users

```sql
CREATE TABLE Users (
    user_id INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    department_id INT NOT NULL,
    role_id INT NOT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    last_login TIMESTAMP NULL,
    FOREIGN KEY (department_id) REFERENCES Departments(department_id),
    FOREIGN KEY (role_id) REFERENCES Roles(role_id)
);
```

Roles

```sql
CREATE TABLE Roles (
    role_id INT PRIMARY KEY AUTO_INCREMENT,
    role_name VARCHAR(50) UNIQUE NOT NULL,
    description TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Insert default roles
INSERT INTO Roles (role_name, description) VALUES
('Admin', 'Risk Management Department with full access'),
('User', 'Department staff with limited access');
```

Departments

```sql
CREATE TABLE Departments (
    department_id INT PRIMARY KEY AUTO_INCREMENT,
    department_name VARCHAR(100) UNIQUE NOT NULL,
    description TEXT,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

2. KRI Module Tables

KRI_Templates

```
CREATE TABLE KRI_Templates (
  template_id INT PRIMARY KEY AUTO_INCREMENT,
  department_id INT NOT NULL,
  created_by INT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NULL ON UPDATE CURRENT_TIMESTAMP,
  is_active BOOLEAN DEFAULT TRUE,
  FOREIGN KEY (department_id) REFERENCES Departments(department_id),
  FOREIGN KEY (created_by) REFERENCES Users(user_id)
);
```

KRI_Indicators

```
CREATE TABLE KRI_Indicators (
  indicator_id INT PRIMARY KEY AUTO_INCREMENT,
  template_id INT NOT NULL,
  rf_no VARCHAR(20) NOT NULL,
  process VARCHAR(255) NOT NULL,
  indicator_description TEXT NOT NULL,
  tolerance_threshold INT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NULL ON UPDATE CURRENT_TIMESTAMP,
  is_active BOOLEAN DEFAULT TRUE,
  FOREIGN KEY (template_id) REFERENCES KRI_Templates(template_id)
);
```

KRI_Submissions

```
CREATE TABLE KRI_Submissions (
  submission_id INT PRIMARY KEY AUTO_INCREMENT,
  department_id INT NOT NULL,
  reporting_month DATE NOT NULL,
  submitted_by INT NOT NULL,
  submission_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  status ENUM('Draft', 'Submitted', 'Reviewed', 'Needs Revision') DEFAULT 'Draft',
  FOREIGN KEY (department_id) REFERENCES Departments(department_id),
  FOREIGN KEY (submitted_by) REFERENCES Users(user_id),
  UNIQUE KEY (department_id, reporting_month)
);
```

KRI_Entries

```
CREATE TABLE KRI_Entries (
    entry_id INT PRIMARY KEY AUTO_INCREMENT,
    submission_id INT NOT NULL,
    indicator_id INT NOT NULL,
    breaches INT NOT NULL,
    risk_tolerance ENUM('Tolerable', 'Breach') GENERATED ALWAYS AS (
        CASE WHEN breaches >= (SELECT tolerance_threshold FROM KRI_Indicators
WHERE indicator_id = indicator_id)
        THEN 'Breach' ELSE 'Tolerable' END
    ) STORED,
    notes TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NULL ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (submission_id) REFERENCES KRI_Submissions(submission_id),
    FOREIGN KEY (indicator_id) REFERENCES KRI_Indicators(indicator_id),
    UNIQUE KEY (submission_id, indicator_id)
);
```

KRI_Attachments

```
CREATE TABLE KRI_Attachments (
    attachment_id INT PRIMARY KEY AUTO_INCREMENT,
    entry_id INT NOT NULL,
    file_name VARCHAR(255) NOT NULL,
    file_path VARCHAR(255) NOT NULL,
    file_size INT NOT NULL,
    file_type VARCHAR(100) NOT NULL,
    uploaded_by INT NOT NULL,
    uploaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (entry_id) REFERENCES KRI_Entries(entry_id),
    FOREIGN KEY (uploaded_by) REFERENCES Users(user_id)
);
```

KRI_Notifications

```
CREATE TABLE KRI_Notifications (
    notification_id INT PRIMARY KEY AUTO_INCREMENT,
    department_id INT NOT NULL,
    notification_type ENUM('Reminder', 'Submission', 'Review', 'Revision') NOT NULL,
    message TEXT NOT NULL,
    is_read BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (department_id) REFERENCES Departments(department_id)
);
```

3. Incident Reporting Module Tables

Incidents

```
CREATE TABLE Incidents (
    incident_id INT PRIMARY KEY AUTO_INCREMENT,
    reported_by INT NOT NULL,
    reporter_title VARCHAR(100) NOT NULL,
    department_id INT NOT NULL,
    report_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    start_date DATE NOT NULL,
    end_date DATE,
    discovery_date DATE NOT NULL,
    discovered_by VARCHAR(100) NOT NULL,
    incident_title VARCHAR(255) NOT NULL,
    activity_process VARCHAR(255) NOT NULL,
    branch_dept_unit VARCHAR(100) NOT NULL,
    risk_owner VARCHAR(100) NOT NULL,
    incident_type ENUM('Loss', 'Near miss', 'Gain', 'Opportunity Cost',
'Undetermined') NOT NULL,
    incident_status ENUM('Open', 'Closed') NOT NULL,
    incident_category ENUM(
        'Internal Fraud',
        'External fraud',
        'Employment Practice and Workplace Safety',
        'Clients, Products and Business Practices',
        'Damage to Physical Assets',
        'Business Disruption and Systems Failure',
        'Execution, Delivery and Process Management'
    ) NOT NULL,
    risk_type ENUM('Process', 'People', 'System', 'External Event') NOT NULL,
    risk_assessment ENUM('Very Low Risk', 'Low Risk', 'Medium Risk', 'High Risk',
'Very High Risk') NOT NULL,
    incident_description TEXT NOT NULL,
    incident_cause TEXT NOT NULL,
    incident_discovery TEXT NOT NULL,
    review_status ENUM('Pending', 'Under Review', 'Revision Requested', 'Approved')
DEFAULT 'Pending',
    review_notes TEXT,
    reviewed_by INT,
    reviewed_at TIMESTAMP NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NULL ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (reported_by) REFERENCES Users(user_id),
    FOREIGN KEY (department_id) REFERENCES Departments(department_id),
    FOREIGN KEY (reviewed_by) REFERENCES Users(user_id)
);
```

Incident_Staff

```
CREATE TABLE Incident_Staff (
    staff_id INT PRIMARY KEY AUTO_INCREMENT,
    incident_id INT NOT NULL,
    staff_name VARCHAR(100) NOT NULL,
    staff_title VARCHAR(100) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (incident_id) REFERENCES Incidents(incident_id)
);
```

Incident_Financial_Impact

```
CREATE TABLE Incident_Financial_Impact (
    impact_id INT PRIMARY KEY AUTO_INCREMENT,
    incident_id INT NOT NULL,
    impact_type ENUM('Loss', 'Gain/Opportunity Cost') NOT NULL,
    currency VARCHAR(3) NOT NULL,
    gross_amount DECIMAL(15,2) NOT NULL,
    legal_costs DECIMAL(15,2),
    other_costs DECIMAL(15,2),
    recovery_from_client DECIMAL(15,2),
    recovery_from_insurance DECIMAL(15,2),
    net_loss DECIMAL(15,2) GENERATED ALWAYS AS (
        gross_amount + COALESCE(legal_costs, 0) + COALESCE(other_costs, 0) -
        COALESCE(recovery_from_client, 0) - COALESCE(recovery_from_insurance, 0)
    ) STORED,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NULL ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (incident_id) REFERENCES Incidents(incident_id)
);
```

Incident_Attachments

```
CREATE TABLE Incident_Attachments (
    attachment_id INT PRIMARY KEY AUTO_INCREMENT,
    incident_id INT NOT NULL,
    file_name VARCHAR(255) NOT NULL,
    file_path VARCHAR(255) NOT NULL,
    file_size INT NOT NULL,
    file_type VARCHAR(100) NOT NULL,
    uploaded_by INT NOT NULL,
    uploaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (incident_id) REFERENCES Incidents(incident_id),
    FOREIGN KEY (uploaded_by) REFERENCES Users(user_id)
);
```

Incident_Revisions

```
CREATE TABLE Incident_Revisions (
    revision_id INT PRIMARY KEY AUTO_INCREMENT,
    incident_id INT NOT NULL,
    requested_by INT NOT NULL,
    requested_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    revision_notes TEXT NOT NULL,
    status ENUM('Pending', 'Completed') DEFAULT 'Pending',
    completed_at TIMESTAMP NULL,
    FOREIGN KEY (incident_id) REFERENCES Incidents(incident_id),
    FOREIGN KEY (requested_by) REFERENCES Users(user_id)
);
```

4. Reporting and Dashboard Tables

Report_Templates

```
CREATE TABLE Report_Templates (
    template_id INT PRIMARY KEY AUTO_INCREMENT,
    template_name VARCHAR(100) UNIQUE NOT NULL,
    template_type ENUM('KRI', 'Incident', 'Combined') NOT NULL,
    template_format ENUM('Excel', 'PDF') NOT NULL,
    template_config JSON NOT NULL,
    created_by INT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NULL ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (created_by) REFERENCES Users(user_id)
);
```

Dashboard_Widgets

```
CREATE TABLE Dashboard_Widgets (
    widget_id INT PRIMARY KEY AUTO_INCREMENT,
    widget_name VARCHAR(100) NOT NULL,
    widget_type ENUM('Chart', 'Table', 'Metric', 'Custom') NOT NULL,
    widget_config JSON NOT NULL,
    created_by INT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NULL ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (created_by) REFERENCES Users(user_id)
);
```

User_Dashboard_Config

```
CREATE TABLE User_Dashboard_Config (
    config_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
    dashboard_layout JSON NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NULL ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);
```

Indexes and Performance Considerations

```
-- Add indexes for frequently queried columns
CREATE INDEX idx_kri_submissions_date ON KRI_Submissions(reporting_month);
CREATE INDEX idx_incidents_dates ON Incidents(report_date, start_date,
discovery_date);
CREATE INDEX idx_incidents_status ON Incidents(incident_status, review_status);
CREATE INDEX idx_incidents_category ON Incidents(incident_category, risk_type,
risk_assessment);
```

Data Retention Policy

```
-- Create a table to track archived data
CREATE TABLE Archived_Data_Log (
    archive_id INT PRIMARY KEY AUTO_INCREMENT,
    data_type ENUM('KRI', 'Incident') NOT NULL,
    archive_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    date_range_start DATE NOT NULL,
    date_range_end DATE NOT NULL,
    archive_location VARCHAR(255) NOT NULL,
    archived_by INT NOT NULL,
    FOREIGN KEY (archived_by) REFERENCES Users(user_id)
);
```

Notes on Implementation

1. **Generated Columns**: The schema uses SQL generated columns for calculated fields like `risk_tolerance` in KRI_Entries and `net_loss` in Incident_Financial_Impact to ensure consistency.

2. **JSON Storage**: Dashboard and report configurations are stored as JSON to allow for flexible customization without schema changes.

3. **Data Validation**: Application-level validation should be implemented to ensure data integrity beyond what database constraints can enforce.

4. **Audit Trail**: Consider implementing triggers or additional audit tables to track changes to critical data.

5. **File Storage**: The schema assumes file attachments are stored on the file system with paths recorded in the database. Consider implementing a secure file storage solution.

6. **Notifications**: The KRI_Notifications table supports the requirement for monthly reminders to departments.

7. **Historical Data**: The schema supports the requirement to maintain data for at least 5 years. Consider implementing an archiving strategy for older data.